

PURS - Laboratorijske vježbe 3 i 4

Matija Holik, 0036516800

Projektiranje filtra u Matlabu

U kontekstu ovog izvještaja projektiran je individualni FIR filter prema zadanim uputama

- Prvo slovo mog imena je M kojem odgovara ASCII kôd 77 stoga je slučajni indeks $I = 77 - 65 = 12$
- Graničnu frekvenciju dobivamo preko izraza $f_q = \frac{\pi}{2} \cdot \frac{I}{25} + \frac{\pi}{4} = \frac{\pi}{2} \cdot \frac{12}{25} + \frac{\pi}{4} = 1.539380$
- Normaliziranu frekvenciju dobivamo podjelom tog izraza s π , odnosno $\frac{f_q}{\pi} = \frac{1.539380}{\pi} = 0.49$
- Studenti projektiraju niskopropusni filter stoga je konačan izraz $\text{fir1}(20, .49, 'low')$
 - o ovdje treba naglasiti da je projektirani filter $N=20$ reda. Taj broj nije uvršten u izračune broja ciklusa po tapu filtra u daljnjem tekstu, no s obzirom da je cilj bio napraviti međusobnu odnosno relativnu usporedbu implementacija to ne bi trebao biti problem.

Pobudni signal također je individualan i dobiva se ovako:

- JMBAG: 0 0 3 6 5 1 6 8 0 0
- odbijeno 5: -5 -5 -2 1 0 -4 1 3 -5 -5
- kroz 7 daje konačni signal $x = [-5/7 -5/7 -2/7 1/7 0 -4/7 1/7 3/7 -5/7 -5/7]$

U Q1.15 reprezentaciji dana pobuda izgleda ovako:

	A	B	C	D	E
1	Vrijednosti u Q1.15 formatu te bazi 10 podjeljene s 2^{15} :				
2	bez zaokruživanja	-0.71429443359375	-0.71429443359375	-0.28570556640625	0.14288330078125
3	s zaokruživanjem	-0.71429443359375	-0.71429443359375	-0.28570556640625	0.1429443359375

F	G	H	I	J	K
0	-0.5714111328125	0.14288330078125	0.4285888671875	-0.71429443359375	-0.71429443359375
0	-0.5714111328125	0.1429443359375	0.4285888671875	-0.71429443359375	-0.71429443359375

Korištenjem tih podataka pomoću Matlab generirani su koeficijenti filtra te potom izvedena konvolucija koristeći pobudni signal kako bi se dobio očekivani izlaz filtra, odnosno našeg programa.

```
Command Window
>> b = fir1(20,.49,'low');
>> bq1 = round(b*2^15);
>> bq = bq1 / 2^15;
>>
>> x = [-5/7 -5/7 -2/7 1/7 0 -4/7 1/7 3/7 -5/7 -5/7]

x =

    -0.7143    -0.7143    -0.2857    0.1429         0    -0.5714    0.1429    0.4286    -0.7143    -0.7143

>> xqi = round(x*2^15);
>> xq = xqi / 2^15;
>> yf = conv(b, x);
>> yf

yf =

Columns 1 through 16

    -0.0006    -0.0030    -0.0015    0.0088    0.0067    -0.0243    -0.0240    0.0585    0.0688    -0.2124    -0.6107    -0.6379    -0.2554    -0.0116    -0.1243    -0.1589

Columns 17 through 30

    0.0427    0.0156    -0.3742    -0.5725    -0.2893    0.0524    0.0880    -0.0183    -0.0343    0.0050    0.0113    -0.0010    -0.0030    -0.0006

>> yi = conv(bq1, xqi);
>> yi

yi =

Columns 1 through 13

   -608556   -3276840   -1647772   9470108   7171554   -26079116   -25835441   62808051   73850321   -228022242   -655755874   -684982201   -274182935

Columns 14 through 26

  -12473902  -133542920  -170641015  45874258   16719395  -401854021  -614688129  -310614787  56249241   94489396  -19670396  -36878271   5420834

Columns 27 through 30

  12138278  -1039242   -3276840   -608556

>> yic = conv(bq, xq);
>> yic

yic =

Columns 1 through 20

    -0.0006    -0.0031    -0.0015    0.0088    0.0067    -0.0243    -0.0241    0.0585    0.0688    -0.2124    -0.6107    -0.6379    -0.2554    -0.0116    -0.1244    -0.1589    0.0427    0.0156    -0.3743    -0.5725

Columns 21 through 30

    -0.2893    0.0524    0.0880    -0.0183    -0.0343    0.0050    0.0113    -0.0010    -0.0031    -0.0006

>> |

UTF-8 CRLF dump_c_data16 Ln 7 Col 22
```

Konačno, te su vrijednosti izvezene u datoteke pogodne za programski jezik C pa možemo napraviti implementaciju samog algoritma. Treba napomenuti da su za usporedbu preciznosti korištene vrijednosti ispisane nakon naredbe 'format long' u Matlabu:

```
y =

Columns 1 through 10

-0.000560672154940 -0.003041412963237 -0.001521609117120 0.008828378661899 0.006676697541326 -0.024285588707423 -0.024049602778138 0.058505435261253 0.068785936774242 -0.212359502327582

Columns 11 through 20

-0.610706619264981 -0.637925143872965 -0.255363471792437 -0.011623175989022 -0.124348243098947 -0.158897544567436 0.042736437626006 0.015580910334455 -0.374246347976751 -0.572454664218384

Columns 21 through 30

-0.289267745727191 0.052383183485552 0.088010630695253 -0.018300338846308 -0.034346783631186 0.005044759264222 0.011309119470196 -0.000960936962180 -0.003041412963237 -0.000560672154940

>> yi

yi =

Columns 1 through 10

-0.000566761940718 -0.003051795065403 -0.001534607261419 0.008819725364447 0.006679030135274 -0.024288069456816 -0.024061129428446 0.058494555763900 0.068778471089900 -0.212362261489034

Columns 11 through 20

-0.610720248892903 -0.637939387001097 -0.255352756939828 -0.011617226526141 -0.124371536076069 -0.158921829424798 0.042723732069135 0.015571149997413 -0.374255721457303 -0.572472930885851

Columns 21 through 30

-0.289282563142478 0.052386187948287 0.088000107556581 -0.018319483846426 -0.034345566295087 0.005048545077443 0.011304652318358 -0.000967869535089 -0.003051795065403 -0.000566761940718
```

Implementacija 1 - floating point

Konačna implementacija generičke implementacije s pomičnim zarezom izgleda ovako:

```
1.  /*******
2.  * fir_float.c
3.  *****/
4.  #include <cycle_count.h>
5.  #include <stdio.h>
6.  #include <ccblkfn.h>
7.
8.  //#define implB 1
9.
10. #define Nsamp 10
11. #define L 21
12.
13. #include "signal_float.h"
14. #include "coeffs_float.h"
15.
16. float delay[L];          // uzorci
17. short idx_delay = 0;     // indeks "najstarijeg" uzorka
18. float izlbuf[Nsamp];    // spremnik izlaznih vrijednosti
19. short idx_izlbuf = 0;   // indeks izlaznih vrijednosti
20.
21. void obrada_float(float sample)
22. {
23.     int i;
24.     float acc = 0;
25.     // zapamti najnoviji uzorak
26.     delay[idx_delay] = sample;
27.
28.     // cirkularno povecaj indeks delay niza (vidi napomenu)
29.     #ifndef implB
30.         idx_delay = (idx_delay+1) % L;
31.     #else
32.         idx_delay = circindex(idx_delay, 1, L); //idx, incr, len
33.     #endif
34.
35.     // u petlji akumuliraj umnoske stanja i koeficijenata, uz cirkularno
36.     // inkrementiranje indeksa stanja (delay)
37.     for(i=0;i<L;i++)
38.     {
39.         acc += delay[idx_delay]*coeffs[i];
40.         #ifndef implB
41.             idx_delay = (idx_delay+1) % L;
42.         #else
43.             idx_delay = circindex(idx_delay, 1, L); //idx, incr, len
44.         #endif
45.     }
46.
47.     izlbuf[idx_izlbuf++] = acc;
48. }
49.
50. int main()
51. {
52.     int i;
53.     cycle_t start_count;
54.     cycle_t final_count;
55.     int final = 0;
56.     for(i=0;i<Nsamp;++i)
57.     {
58.         START_CYCLE_COUNT(start_count);
59.         obrada_float(signal[i]);
60.         STOP_CYCLE_COUNT(final_count, start_count);
61.         PRINT_CYCLES("Number of cycles: ", final_count);
62.         final += final_count;
63.         printf("%f\n", izlbuf[i]);
```

```

64.     }
65.
66.     printf("final=%d\n", final);
67.     return 0;
68. }

```

te su pripadni ispisi A i B inačice sljedeći:

<pre> Loading: "C:\mh\Documents\VisualDSP Projects\fir_float\Release\fir_float.dxe"... Load complete. Breakpoint Hit at <ffa1130c> Number of cycles: 2030 -0.000561 Number of cycles: 2098 -0.003041 Number of cycles: 2166 -0.001522 Number of cycles: 2234 0.008828 Number of cycles: 2234 0.006677 Number of cycles: 2302 -0.024286 Number of cycles: 2370 -0.024050 Number of cycles: 2438 0.058505 Number of cycles: 2506 0.068786 Number of cycles: 2574 -0.212360 final=22952 Breakpoint Hit at <ffa12f02> </pre>	<pre> Loading: "C:\mh\Documents\VisualDSP Projects\fir_float\Release\fir_float.dxe"... Load complete. Breakpoint Hit at <ffa112d0> Number of cycles: 1686 -0.000561 Number of cycles: 1758 -0.003041 Number of cycles: 1830 -0.001522 Number of cycles: 1902 0.008828 Number of cycles: 1902 0.006677 Number of cycles: 1974 -0.024286 Number of cycles: 2046 -0.024050 Number of cycles: 2118 0.058505 Number of cycles: 2190 0.068786 Number of cycles: 2262 -0.212360 final=19668 Breakpoint Hit at <ffa12eca> </pre>
--	--

Odgovori na pitanja iz uputa

1. Kako je cirkularno indeksiranje izvedeno u assembleru u prvoj a kako u drugoj varijanti?
 - a. prva implementacija implementira indeksiranje po matematičkoj definiciji, odnosno korištenjem modulo operacije. Druga varijanta koristi uvjetnu logiku te osnovne operacije inkrementiranja i pridruživanja nakon što pređemo kraj.
2. Kako je izvedeno množenje i akumulacija?
 - a. Množenje i akumulacija koriste gotove metode standardne biblioteke koje emuliraju rad s pomičnim zarezom, konkretno `__float32_mul` i `__float32_add` jer je procesor cjelobrojni, tj. ne može direktno raditi s brojevima s pomičnim zarezom.
3. Kako je izvedena for petlja?
 - a. U Debug profilu For petlja je također izvedena klasičnim pristupom: brojač se inkrementira za jedan i oduzima od zadane konačne vrijednosti te se izvodi skok na početak petlje. Tamo se provjerava uvjet petlje manje ili jednako logičkom operacijom i ako nije zadovoljen skokom se izlazi iz petlje, što je izvedeno korištenjem uvjetnih instrukcija. U Release profilu koristi se sklopovska izvedba petlje, tj. ne vide se eksplicitne provjere uvjeta i skokovi već samo čitanje brojača.
4. Koji je broj ciklusa za svaku od varijanti?
 - a. Zbog malog broja uzorka „podatkovni cjevovod“ se ne stigne u potpunosti napuniti, ali možemo usporediti relativne performanse: varijanta A odrađuje svoj posao u 22952 ciklusa dok varijanta B treba 19668
5. Koliki je broj ciklusa po jednom tapu filtra (po jednom uzorku i koeficijentu, odnosno po jednom prolazu petlje u funkciji obrada float)?
 - a. Uz istu napomenu kao i za prethodno pitanje, najbolja procjena je da varijanta A zahtjeva 2574 ciklusa a varijanta B 2262
6. Koja je najveća dopuštena frekvencija otipkavanja signala koji obrađujemo ovim filtrom, ako je takt procesora podešen na 300 MHz?
 - a. Ako procesor obavlja F ciklusa u sekundi, a potrebno je O ciklusa po jednom uzorku, broj uzoraka N koje procesor može obaviti je: $N = F/O$. Ako gledamo B varijantu, obrada jednog uzorka traje oko $O=2262$. stoga je maksimalan broj uzoraka u sekundi $N=300 \text{ MHz} / 2262 = 132.6 * 10^3$

Što se tiče pitanja preciznosti, naš program koristi single precision IEEE 754 normu, dok Matlab koristi double precision stoga se očekuje razlika reda veličine 10^{-6} . Usporedba stvarni vrijednosti potvrđuje to očekivanje.

Implementacija 2 - generička cjelobrojna

```
1.  /*****
2.  * fir_integer.c
3.  *****/
4.  #include <cycle_count.h>
5.  #include <stdio.h>
6.  #include <ccblkfn.h>
7.  #include "signal_integer.h"
8.  #include "coeffs_integer.h"
9.
10. #define L 21 /* broj uzoraka impulsnog odziva */
11. #define Nsamp 10
12.
13. short delay[L];
14. short idx_delay = 0;
15. short izlbuf[Nsamp];
16. short idx_izlbuf = 0;
17.
18. void obrada_integer(short sample)
19. {
20.     int i;
21.     int acc = 0;
22.
23.     // zapamti najnoviji uzorak
24.     delay[idx_delay] = sample;
25.
26.     // cirkularno povecaj indeks delay niza (circindex)
27.     idx_delay = circindex(idx_delay, 1, L);
28.
29.     // u petlji akumuliraj umnoske stanja i koeficijenata, uz cirkularno
30.     // inkrementiranje indeksa stanja pomocu circindex
31.     // voditi racuna o specifcnostima fixed point aritmetike, odnosno
32.     // posmaku u lijevo za jedan nakon mnozenja da bi rezultat bio tocan
33.     // u 32 bitnoj frakcionalnoj aritmetici (jeste li proucili pripremu?)
34.     // zadrzi samo gornjih 16 bitova akumulatora (za sad ne brinemo oko
35.     // overflowa) i posmakni na najmanje znacajna dva bajta (>>16)
36.     for (i = 0; i < L; i++)
37.     {
38.         acc += ((int)((int)delay[idx_delay] * (int)coeffs[i]<<1)>>16; // pomnozi, ukloni dupli bit
predznaka, zaokruzi i zadrzi gornjih 16 bitova
39.         idx_delay = circindex(idx_delay, 1, L);
40.     }
41.
42.     izlbuf[idx_izlbuf++] = acc;
43. }
44.
45. int main()
46. {
47.     int i;
48.     cycle_t start_count;
49.     cycle_t final_count;
50.     for(i=0;i<Nsamp;++i)
51.     {
52.         START_CYCLE_COUNT(start_count);
53.         obrada_integer(signal[i]);
54.         STOP_CYCLE_COUNT(final_count, start_count);
55.         printf("Number of cycles: %d\n", final_count);
56.         printf("%d\n", izlbuf[i]); //za provjeru ispis/2e15
57.     }
58.     return 0;
59. }
```

te je pripadni ispis sljedeći:

```
Loading: "C:\mh\Documents\VisualDSP Projects\fir_integer\Release\fir_integer.dxe"...
Load complete.
Breakpoint Hit at <ffa112c4>

Number of cycles: 356
-19
Number of cycles: 366
-101
Number of cycles: 358
-52
Number of cycles: 348
287
Number of cycles: 354
217
Number of cycles: 364
-798
Number of cycles: 360
-791
Number of cycles: 350
1913
Number of cycles: 352
2250
Number of cycles: 362
-6962

Breakpoint Hit at <ffa12e9a>
```

Podjelom ovih brojeva s 2^{15} dobivamo očekivane vrijednosti koje prikazuju Matlab i prethodni primjer, no s većom greškom: razlike primjećujemo već na petoj decimali, no uz gotovo reda veličine bolje performanse to može biti prihvatljiv kompromis za određene situacije.

Odgovori na pitanja iz uputa

1. Kako je cirkularno indeksiranje izvedeno u assembleru?
 - a. I dalje se koristi circindex naredba stoga je odgovor isti.
2. Kako je izvedeno množenje i akumulacija?
 - a. Ove su operacije sada izvedene direktno, koristeći osnovne operacije zbrajanja, množenja i posmaka. Gledajući u asemblerski kôd i procjene broja ciklusa potrebne za simulacija operacija nad brojevima s pomičnom točkom, implementacija s cijelim brojevima sadrži znatno manje instrukcija za svaki prolaz što se i očituje u konačnom broju ciklusa. Primjerice, funkcija `__float32_mul` ima preko 90 asemblerskih instrukcija dok se isti zadatak u cjelobrojnom računu obavi u jednom, ne računajući dohvat argumenata.
3. Kako je izvedena for petlja?
 - a. Ne vidim neke značajnije razlike od prošle implementacije
4. Koji je broj ciklusa za ovu implementaciju?
 - a. Uz istu napomenu kao i za prethodnu implementaciju, ova odrađuje svoj posao u 3570 ciklusa
5. Koliki je broj ciklusa po jednom tapu filtra?
 - a. Uz istu napomenu kao i za prethodno pitanje, najbolja procjena je je potrebno oko 357 ciklusa.
6. Koja je najveća dopuštena frekvencija otipkavanja signala koji obrađujemo ovim filtrom, ako je takt procesora podešen na 300 MHz?
 - a. Po istoj logici kao i za prošlu implementaciju, tj. $N = F/O$ te $O=357$ slijedi da je $N=300 \text{ MHz} / 357 = 840.3 \cdot 10^3$

Implementacija 3 - frakcionalni tipovi i built-in funkcije

```
1.  /*****
2.  * fir_fract16.c
3.  *****/
4.  #include <fract.h>
5.  #include <cycle_count.h>
6.  #include <stdio.h>
7.  #include <ccblkfn.h>
8.  #include "signal_fract16.h"
9.  #include "coeffs_fract16.h"
10.
11. #define implB 1
12.
13. #define L 21 /* broj uzoraka impulsnog odziva */
14. #define Nsamp 10
15.
16. fract16 delay[L];
17.
18. #ifndef implB
19. short idx_delay = 0;
20. #else
21. int idx_delay = 0;
22. #endif
23.
24. fract16 izlbuf[Nsamp];
25. short idx_izlbuf = 0;
26.
27. void obrada_fract16(fract16 sample)
28. {
29.     int i;
30.     fract32 acc = 0;
31.     fract32 prod;
32.
33.     // zapamti najnoviji uzorak
34.     delay[idx_delay] = sample;
35.
36.     // cirkularno povecaj indeks delay niza (circindex)
37.     idx_delay = circindex(idx_delay, 1, L);
38.
39.     // kao i u prethodnim zadacima implementirajte mnozenje i akumulaciju
40.     // ovaj put koristeci fract16 i fract32 tipove podataka.
41.     // da bi to napravili, koristite naredbe mult_fr1x32, add_fr1x32 i
42.     // round_fr1x32
43.     for (i = 0; i < L; i++)
44.     {
45.         acc += round_fr1x32(add_fr1x32(acc, mult_fr1x32(delay[idx_delay], coeffs[i])));
46.         idx_delay = circindex(idx_delay, 1, L);
47.     }
48.
49.     izlbuf[idx_izlbuf++] = acc;
50. }
51.
52. int main()
53. {
54.     int i;
55.     cycle_t start_count;
56.     cycle_t final_count;
57.
58.     for(i=0;i<Nsamp;++i)
59.     {
60.         START_CYCLE_COUNT(start_count);
61.         obrada_fract16(signal[i]);
62.         STOP_CYCLE_COUNT(final_count, start_count);
63.         printf("Number of cycles: %d\n", final_count);
64.         printf("%d\n", izlbuf[i]);
```

```

65.     }
66.     return 0;
67. }

```

te su pripadni ispisi A i B inačice sljedeći:

<pre> Loading: "C:\mh\Documents\VisualDSP Projects\fir_fract16\Release\fir_fract16.dxe"... Load complete. Breakpoint Hit at <ffa112d0> Number of cycles: 398 -19 Number of cycles: 408 -100 Number of cycles: 400 -49 Number of cycles: 390 289 Number of cycles: 396 218 Number of cycles: 406 -796 Number of cycles: 402 -788 Number of cycles: 392 1917 Number of cycles: 394 2253 Number of cycles: 404 -6961 Breakpoint Hit at <ffa12ea2> </pre>	<pre> Loading: "C:\mh\Documents\VisualDSP Projects\fir_fract16\Release\fir_fract16.dxe"... Load complete. Breakpoint Hit at <ffa112f0> Number of cycles: 154 -19 Number of cycles: 155 -100 Number of cycles: 155 -49 Number of cycles: 154 289 Number of cycles: 154 218 Number of cycles: 155 -796 Number of cycles: 155 -788 Number of cycles: 154 1917 Number of cycles: 154 2253 Number of cycles: 155 -6961 Breakpoint Hit at <ffa12ec2> </pre>
---	---

Kao što se može iščitati iz priloženih slika, performanse frakcionalne implementacije su 16 puta bolje od implementacije s pomičnim zarezom te dva do tri puta bolje od cjelobrojne implementacije uz usporedivu preciznost. „Ispod haube“, frakcionalni račun svodi se na cjelobrojni, no omogućava korištenje optimiziranih ugrađenih procedura koje su ujedno i inline pa se i štedi vrijeme jer nema funkcijskog poziva. Konačno, provodilac može optimalnije koristiti registre s njima.

Odgovori na pitanja iz uputa

1. Kako je cirkularno indeksiranje izvedeno u assembleru?
 - a. Implementacija B, za razliku od prijašnjih, izvodi inkrementiranje sklopovski, konkretnije pomoću DAG sklopova što u assembleru izgleda kao postavljanje i kasnije čitanje Ix, Ly i Mz registara. U mom primjeru prevodilac je odabrao M0 te I1 i L1. Neki od tih registara koriste se i u A inačici, ali ne u kombinaciji s DAGom.
 - b. Kako bi se ove promjene vidjele bilo je potrebno prebaciti prevoditelja s Debug na Release profil.
2. Kako je izvedeno množenje i akumulacija?
 - a. Ove su procedure obavljene koristeći ugrađene procedure add_fr1x32 i mult_fr1x32 koje se mapiraju na gotove assemblyske instrukcije
3. Kako je izvedena for petlja?
 - a. Ne vidim neke značajnije razlike od prošle implementacije
4. Koji je broj ciklusa za ovu implementaciju?
 - a. Uz istu napomenu kao i za prethodnu implementaciju, ova odrađuje svoj posao u 1545 ciklusa
5. Koliki je broj ciklusa po jednom tapu filtra?
 - a. Uz istu napomenu kao i za prethodno pitanje, najbolja procjena je je potrebno oko 155 ciklusa.
6. Koja je najveća dopuštena frekvencija otipkavanja signala koji obrađujemo ovim filtrom, ako je takt procesora podešen na 300 MHz?
 - a. Po istoj logici kao i za prošlu implementaciju, tj. $N = F/O$ te $O=155$ slijedi da je $N=300 \text{ MHz} / 155 = 1.84 \cdot 10^6$

Implementacija 4 - ugrađene funkcije

```
1.  /*****
2.  * fir_fr16.c
3.  *****/
4.  #include <fract.h> #include <cycle_count.h> #include <stdio.h> #include <ccblkfn.h>
5.  #include <filter.h> #include "signal_fract16.h" #include "coeffs_fract16.h"
6.
7.  #define L 21
8.  #define Nsamp 10
9.
10. fract16 delay[L];
11. int idx_delay = 0;
12. fract16 izlbuf[Nsamp];
13. short idx_izlbuf = 0;
14.
15. void obrada_fir_fr16(void)
16. {
17.     fir_state_fr16 filter_state;
18.
19.     // initialize the filter_state struc
20.     fir_init(filter_state, coeffs, delay, L, 0);
21.
22.     // run the operation
23.     fir_fr16(signal, izlbuf, Nsamp, &filter_state);
24. }
25.
26. int main()
27. {
28.     int i,j;
29.     cycle_t start_count;
30.     cycle_t final_count;
31.
32.     START_CYCLE_COUNT(start_count);
33.     obrada_fir_fr16();
34.     STOP_CYCLE_COUNT(final_count, start_count);
35.     printf("Number of cycles: %d\n", final_count);
36.
37.     for(i=0; i<Nsamp; ++i)
38.     {
39.         printf("%d\n", izlbuf[i]);
40.     }
41.     return 0;
42. }
```

te je pripadni ispis sljedeći:

```
Loading: "C:\mh\Documents\VisualDSP Projects\fir_fr16\Release\fir_fr16.dxe"...
Load complete.
Breakpoint Hit at <ffaf1408>

Number of cycles: 278
-19
-100
-50
289
219
-796
-788
1917
2254
-6959

Breakpoint Hit at <ffaf2fda>
```

Iako također rade s fract16 formatom podataka, ugrađene funkcije za rad FIR filtra koriste svega 278 ciklusa za obradu svih 10 uzoraka, što je red veličine bolje od onoga što smo vidjeli do sada! Drugim riječima, broj ciklusa po tapu $O=27.8 \sim 28$ što znači da bismo pri radnoj frekvenciji od $F=300$ MHz mogli otipkavati $N = F/O = 10.7 * 10^6$ uzoraka u sekundi. Kompletan asemblerski kod sadrži svega 40 instrukcija, dobar dio kojih u stvari samo konfigurira sklopovlje za izvedbu petlje te adresne generatore. Mogu se primijetiti i funkcije s višestrukim dohvatom.